

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN THIẾT KẾ LUẬN LÝ

BÁO CÁO ĐỒ ÁN

Hệ thống IoT RTOS giám sát chất lượng không khí (IAQ)

Học kỳ 2, năm học 2025–2026

Giảng viên hướng dẫn Phan Văn Sỹ
Lớp L02

Sinh viên	MSSV
Hồ Công Danh	2310410
Hoàng Anh Duy	2310458
Cao Quang Đôn	2310752

TP. Hồ Chí Minh, tháng 05 năm 2026

Tóm tắt

Đề tài xây dựng một hệ thống IoT RTOS giám sát và điều chỉnh chất lượng không khí trong nhà (IAQ). Hệ thống gồm thiết bị nhúng ESP32, firmware đọc cảm biến, tầng RTOS/IoT truyền dữ liệu, backend server, cơ sở dữ liệu và dashboard web.

Phần firmware hiện tại đọc bụi mịn PMS7003 qua UART, CO₂ và nhiệt độ/độ ẩm qua RS485 Modbus, VOC qua ADC, sau đó chuẩn hóa dữ liệu thành **SensorSample**. Dữ liệu được đánh giá bởi **IaqEvaluator** theo vùng an toàn/cần can thiệp/cảnh báo, áp dụng hysteresis và hold-time để giảm dao động, rồi điều khiển LED hoặc thiết bị mô phỏng qua **IaqController**. Hệ thống cũng ghi CSV vào SPIFFS để phục vụ kiểm thử.

Các phần RTOS, MQTT, backend, database và dashboard được thiết kế theo hướng mở rộng: ESP32 publish telemetry lên MQTT, backend nhận dữ liệu và cung cấp API, database lưu lịch sử, dashboard hiển thị realtime/history và gửi lệnh điều khiển ngược xuống thiết bị.

Mục lục

Tóm tắt	i
1 Giới thiệu đề tài	1
1.1 Bối cảnh	1
1.2 Mục tiêu	2
1.3 Phạm vi và hiện trạng	3
2 Yêu cầu và cơ sở lý thuyết	4
2.1 Yêu cầu và cơ sở lý thuyết	4
2.1.1 Các đại lượng giám sát IAQ	4
2.1.2 Định hướng giám sát và cảnh báo	5
2.1.3 Yêu cầu chức năng	5
2.1.4 Yêu cầu phi chức năng	6
3 Kiến trúc hệ thống	7
3.1 Luồng dữ liệu	7
3.2 Luận điểm thiết kế hệ thống	8
3.3 Phân rã module	8
3.4 Nguyên tắc thiết kế	9
4 Thiết kế phần cứng	11
4.1 Thiết kế phần cứng	11
4.1.1 Nền tảng điều khiển	11
4.1.2 Khối nguồn	13
4.1.3 Cảm biến và đặc tính kết nối	13
4.1.4 Lý do lựa chọn giao tiếp cảm biến	14
4.1.5 Thiết bị chấp hành và cảnh báo	14
5 Firmware đọc dữ liệu và điều khiển cục bộ	16
5.1 Firmware đọc dữ liệu và xử lý cục bộ	16
5.1.1 Vai trò của firmware	16
5.1.2 Mô hình dữ liệu trung tâm	16

5.1.3	Đọc cảm biến và chuẩn hóa dữ liệu	17
5.1.4	Đánh giá IAQ và cơ chế hysteresis	18
5.1.5	Bảng nhân quả vận hành hệ thống	18
5.1.6	Áp dụng điều khiển và lưu dữ liệu	19
6	Thiết kế RTOS và IoT truyền dữ liệu	20
6.1	Thiết kế RTOS và tích hợp IoT	20
6.1.1	Mục tiêu tổ chức RTOS	20
6.1.2	Khởi tạo hệ thống	20
6.1.3	Tài nguyên đồng bộ	21
6.1.4	Phân rã task trong firmware	22
6.1.5	Luồng telemetry từ ESP32 lên backend	22
6.1.6	MQTT contract và luồng điều khiển	23
6.1.7	Cấu hình ngưỡng và vận hành khi mất kết nối	24
7	Server, cơ sở dữ liệu và dashboard web	25
7.1	Backend server	25
7.1.1	Thiết kế lưu trữ dữ liệu	27
7.1.2	Mô hình dữ liệu realtime và lịch sử	27
7.1.3	Dashboard web	28
7.1.4	Luồng điều khiển và truy vết	29
7.1.5	Contract dữ liệu giữa các tầng	29
8	Kết luận và hướng phát triển	31
8.1	Kết luận	31
8.2	Giới hạn hiện tại	31
8.3	Hướng phát triển	32
	Tài liệu tham khảo	34

1. Giới thiệu đề tài

Đề tài tập trung xây dựng một hệ thống IoT giám sát và hỗ trợ điều chỉnh chất lượng không khí trong phòng học. Hệ thống sử dụng ESP32 làm nút trung tâm để thu thập dữ liệu từ các cảm biến môi trường, đánh giá trạng thái chất lượng không khí theo thời gian thực và phát cảnh báo khi các thông số vượt ngưỡng cho phép.

Dữ liệu từ thiết bị được truyền về máy chủ để lưu trữ, xử lý và hiển thị trên dashboard web. Thông qua giao diện này, người dùng có thể theo dõi các thông số môi trường, xem dữ liệu lịch sử và gửi lệnh điều khiển thiết bị từ xa. Ngoài ra, hệ thống cũng hỗ trợ điều khiển cục bộ nhằm mô phỏng hoạt động của các thiết bị cải thiện chất lượng không khí như quạt thông gió, máy lọc không khí hoặc đèn cảnh báo.

Với cách tiếp cận này, đề tài hướng đến việc xây dựng một mô hình hệ thống IoT hoàn chỉnh, bao gồm các chức năng thu thập dữ liệu, truyền thông, lưu trữ, giám sát và điều khiển, phù hợp với bài toán theo dõi chất lượng không khí trong môi trường phòng học.

1.1 Bối cảnh

Phòng học là không gian mà học sinh và giáo viên sử dụng trong phần lớn thời gian trong ngày, vì vậy chất lượng không khí trong nhà (Indoor Air Quality – IAQ) ảnh hưởng trực tiếp đến sức khỏe, mức độ tập trung và hiệu quả học tập. Theo Tổ chức Y tế Thế giới, nhiều chất ô nhiễm trong nhà như carbon monoxide, formaldehyde, nitrogen dioxide, benzene và một số hợp chất hữu cơ có thể gây rủi ro đáng kể đối với sức khỏe khi xuất hiện trong không gian kín ở nồng độ không mong muốn [1]. Đối với môi trường học đường, Cơ quan Bảo vệ Môi trường Hoa Kỳ cũng chỉ ra rằng IAQ kém có thể gây triệu chứng sức khỏe cấp tính, làm tăng tình trạng vắng học và làm suy giảm khả năng thực hiện các nhiệm vụ trí óc như tập trung, tính toán và ghi nhớ [2].

Nhiều nghiên cứu cho thấy điều kiện thông gió kém, nồng độ CO₂ cao và sự tích tụ bụi mịn có liên quan đến sự suy giảm hiệu quả học tập. Pulimeno và cộng sự nhấn mạnh rằng lớp học đông người, quá nóng và thông gió kém thường dẫn đến gia tăng CO₂ cùng các vấn đề về sức khỏe, sự thoải mái và hiệu suất học tập [3]. Trong một tổng quan định lượng, Wargoeki và cộng sự ước tính rằng việc giảm nồng độ CO₂ trong lớp học từ khoảng 2100 ppm xuống 900 ppm có thể cải thiện tốc độ thực hiện các bài kiểm tra và nhiệm

vụ học tập khoảng 12%, đồng thời giảm tỷ lệ lỗi khoảng 2% [4]. Khảo sát của Canha và cộng sự tại 9 phòng học cũng cho thấy nhiều tiết học vượt ngưỡng khuyến nghị về nhiệt độ, PM10 và CO₂, qua đó cho thấy nhu cầu giám sát liên tục trong môi trường lớp học [5].

Trong bối cảnh đó, các hệ thống IoT có thể hỗ trợ đo lường, lưu trữ, phân tích và theo dõi chất lượng không khí theo thời gian thực. Các nghiên cứu gần đây đã đề xuất việc áp dụng IoT và học máy vào quản lý chất lượng không khí lớp học [6], đồng thời cho thấy các chiến lược thông gió điều khiển theo nhu cầu có thể giúp giảm vận hành không cần thiết và tiết kiệm năng lượng [7]. Vì vậy, việc xây dựng một hệ thống giám sát và cảnh báo IAQ sử dụng cảm biến và IoT là một hướng tiếp cận phù hợp cho môi trường phòng học.

1.2 Mục tiêu

Mục tiêu của đề tài là thiết kế và xây dựng một hệ thống IoT giám sát chất lượng không khí trong phòng học, có khả năng thu thập dữ liệu từ nhiều loại cảm biến, xử lý trên vi điều khiển ESP32, truyền dữ liệu lên máy chủ và hiển thị trực quan trên dashboard web. ESP32 phù hợp với hướng tiếp cận này nhờ tích hợp kết nối không dây và đủ tài nguyên để xử lý nhiều giao tiếp cảm biến trên cùng một thiết bị [11]; trong khi đó, MQTT là giao thức publish/subscribe nhẹ, phù hợp với luồng dữ liệu giữa thiết bị và backend trong hệ thống IoT [12, 13].

Cụ thể, đề tài hướng đến các mục tiêu sau:

- Thiết kế phần cứng đo IAQ trong phòng học dựa trên ESP32 và các cảm biến phổ biến, phục vụ thu thập CO₂, bụi mịn, nhiệt độ, độ ẩm và chỉ số VOC.
- Xây dựng firmware nhúng cho ESP32 có khả năng đọc dữ liệu từ nhiều cảm biến, chuẩn hóa mẫu đo và phát hiện các trạng thái bất thường như lỗi đọc, mất kết nối hoặc dữ liệu thiếu.
- Tổ chức firmware theo kiến trúc RTOS, trong đó các chức năng đọc cảm biến, xử lý dữ liệu, điều khiển thiết bị và truyền thông MQTT được tách thành các tác vụ độc lập.
- Xây dựng luồng truyền thông IoT từ thiết bị lên backend, bao gồm tiếp nhận dữ liệu qua MQTT, lưu trữ lịch sử và cung cấp API cho dashboard.
- Thiết kế dashboard web cho phép theo dõi dữ liệu realtime, xem lại lịch sử đo, quan sát trạng thái hệ thống và gửi lệnh điều khiển xuống thiết bị.
- Đánh giá hoạt động của hệ thống ở mức nguyên mẫu thông qua khả năng đo, truyền, lưu trữ, hiển thị và phản hồi lệnh điều khiển.

1.3 Phạm vi và hiện trạng

Báo cáo tập trung vào nguyên mẫu gồm một nút ESP32 thu thập dữ liệu từ các cảm biến chất lượng không khí, xử lý cục bộ và truyền telemetry lên hệ thống máy chủ. Ở phía server, backend tiếp nhận dữ liệu qua MQTT, lưu lịch sử vào MySQL và cung cấp API cho dashboard web. Trọng tâm của báo cáo là mô tả kiến trúc hệ thống, luồng dữ liệu, cơ chế xử lý trên firmware RTOS và sự phối hợp giữa các thành phần nhúng, backend và giao diện người dùng.

Ở thời điểm hiện tại, hệ thống đã đạt mức nguyên mẫu vận hành tương đối đầy đủ: firmware RTOS đã được tổ chức theo mô hình đa tác vụ; contract MQTT/JSON giữa thiết bị và backend đã được xác định; backend và cơ sở dữ liệu đã hỗ trợ lưu trữ và truy xuất dữ liệu; còn dashboard đã có các chức năng giám sát và điều khiển cơ bản. Tuy nhiên, phạm vi hiện tại vẫn dừng ở mức kiểm chứng kỹ thuật và chưa mở rộng sang các yêu cầu triển khai thực địa như thiết bị chấp hành công suất thực, vận hành dài hạn trong phòng học thật, hay quản trị nhiều thiết bị ở quy mô lớn.

Bảng 1.1: Hiện trạng các thành phần chính của nguyên mẫu

Thành phần	Vai trò	Hiện trạng
rtos	Firmware RTOS cho ESP32, xử lý cảm biến, cảnh báo, MQTT và điều khiển	Đã hiện thực
common	Contract MQTT topic và payload JSON dùng chung	Đã xác định
iot_backend/backend	Tiếp nhận MQTT, cung cấp API và xử lý điều khiển	Đã hiện thực
iot_backend/database	Lưu telemetry và nhật ký điều khiển	Đã có schema
iot_backend/frontend	Dashboard giám sát và control panel	Đã hiện thực giao diện
integration_test	Kiểm thử tích hợp một phần pipeline dữ liệu và điều khiển	Đã có một phần

Một giới hạn quan trọng của hiện trạng hiện nay là hệ thống vẫn cần được kiểm thử end-to-end đầy đủ với ESP32 thật, broker thật, cơ sở dữ liệu thật và dashboard vận hành đồng thời trong điều kiện sử dụng liên tục. Vì vậy, báo cáo phân biệt rõ giữa phần đã được hiện thực và kiểm chứng ở mức nguyên mẫu trong source code với phần còn cần đánh giá thêm khi triển khai thực nghiệm trong môi trường thực tế.

2. Yêu cầu và cơ sở lý thuyết

2.1 Yêu cầu và cơ sở lý thuyết

2.1.1 Các đại lượng giám sát IAQ

Trong phạm vi đề tài, hệ thống tập trung theo dõi năm nhóm đại lượng chính gồm CO₂, bụi mịn, VOC, nhiệt độ và độ ẩm. Các đại lượng này được lựa chọn vì phản ánh đồng thời tình trạng thông gió, mức độ ô nhiễm không khí và điều kiện tiện nghi trong phòng học.

CO₂ được sử dụng như một chỉ báo gián tiếp của mức độ thông gió và mật độ người trong phòng. Khi lớp học đông người hoặc thông gió kém, nồng độ CO₂ có xu hướng tăng dần, do đó việc theo dõi đại lượng này giúp hệ thống nhận biết thời điểm cần tăng thông gió hoặc phát cảnh báo. Đây cũng là chỉ báo được sử dụng phổ biến trong các tài liệu hướng dẫn IAQ cho trường học [8, 9].

Bụi mịn được biểu diễn qua các chỉ số PM1.0, PM2.5 và PM10. Trong hệ thống này, cảm biến PMS7003 có khả năng đọc cả ba chỉ số, tuy nhiên pipeline IoT hiện tại ưu tiên sử dụng PM2.5 làm chỉ số bụi chính để publish MQTT, lưu cơ sở dữ liệu và hiển thị trên dashboard. PM2.5 được lựa chọn vì liên quan trực tiếp đến nguy cơ sức khỏe và thường được dùng trong các khuyến nghị chất lượng không khí [10].

VOC phản ánh sự hiện diện của các hợp chất hữu cơ bay hơi phát sinh từ vật liệu xây dựng, sơn, dung môi, hóa chất vệ sinh hoặc hoạt động sinh hoạt trong phòng. Một số hợp chất thuộc nhóm này đã được WHO xếp vào nhóm chất ô nhiễm trong nhà cần quan tâm [1]. Trong nguyên mẫu hiện tại, giá trị VOC được đọc ở dạng *raw/index* tương đối, do đó chủ yếu được dùng để theo dõi xu hướng và kích hoạt logic cảnh báo ở mức prototype.

Nhiệt độ và độ ẩm không trực tiếp đại diện cho nồng độ chất ô nhiễm, nhưng ảnh hưởng rõ rệt đến tiện nghi nhiệt, cảm nhận môi trường và điều kiện vận hành của lớp học. Vì vậy, hai đại lượng này được đưa vào hệ thống nhằm hỗ trợ đánh giá tổng thể điều kiện môi trường và làm cơ sở cho các quyết định điều khiển cục bộ.

2.1.2 Định hướng giám sát và cảnh báo

Mục tiêu của hệ thống không phải là thay thế thiết bị đo kiểm chuyên dụng hay đưa ra kết luận y tế tuyệt đối về chất lượng không khí. Đề tài hướng đến việc xây dựng một nguyên mẫu IoT có khả năng giám sát liên tục, phát hiện xu hướng bất thường và hỗ trợ điều khiển cục bộ. Vì vậy, dữ liệu đo được được sử dụng theo hai hướng chính: hiển thị trạng thái môi trường theo thời gian thực và kích hoạt cảnh báo hoặc thiết bị chấp hành khi đại lượng vượt khỏi vùng vận hành mong muốn.

Đối với CO₂, PM2.5, nhiệt độ và độ ẩm, hệ thống có thể so sánh trực tiếp với các ngưỡng cấu hình trong firmware. Đối với VOC, do giá trị hiện tại mang tính tương đối, hệ thống chủ yếu dùng để quan sát xu hướng và kích hoạt cảnh báo ở mức nguyên mẫu. Cách tiếp cận này phù hợp với phạm vi đồ án, vốn tập trung vào kiến trúc đo, truyền dữ liệu, xử lý lỗi, hiển thị và điều khiển hơn là chứng nhận IAQ theo tiêu chuẩn y tế hoặc công trình.

Trong firmware RTOS, các cặp ngưỡng bật/tắt như `co2_on/co2_off`, `pm_on/pm_off`, `voc_on/voc_off`, `temp_high_on/temp_high_off` và các ngưỡng độ ẩm thấp/cao được dùng cùng cơ chế hysteresis để tránh thiết bị bật tắt liên tục khi giá trị đo dao động gần ngưỡng. Các tham số này có thể được hiệu chỉnh trong quá trình kiểm thử thông qua backend hoặc giao diện điều khiển.

2.1.3 Yêu cầu chức năng

Từ mục tiêu giám sát IAQ và điều khiển thiết bị trong phòng học, hệ thống cần đáp ứng các yêu cầu chức năng sau:

1. ESP32 đọc dữ liệu cảm biến theo chu kỳ và tạo mẫu đo có dấu thời gian tương đối trong quá trình vận hành.
2. Firmware thu thập được dữ liệu từ nhiều cảm biến sử dụng các giao tiếp khác nhau, bao gồm cảm biến bụi, CO₂, VOC, nhiệt độ và độ ẩm.
3. Dữ liệu cảm biến được chuẩn hóa về một mô hình telemetry thống nhất trước khi đưa vào các khối xử lý, điều khiển hoặc truyền thông MQTT.
4. Firmware biểu diễn được trạng thái dữ liệu thiếu, lỗi đọc hoặc mất kết nối để tránh sử dụng giá trị không đáng tin cậy cho quyết định điều khiển.
5. Hệ thống đánh giá từng nhóm đại lượng IAQ dựa trên ngưỡng cấu hình và xác định nguyên nhân gây cảnh báo.
6. Hệ thống điều khiển thiết bị cục bộ theo nguyên nhân phát hiện được, chẳng hạn thông gió cho CO₂/VOC, lọc bụi cho PM2.5, điều hòa cho nhiệt độ hoặc tạo ẩm khi độ ẩm thấp.

7. ESP32 gửi telemetry lên backend thông qua MQTT theo định dạng JSON thống nhất.
8. Backend tiếp nhận telemetry, kiểm tra định dạng dữ liệu, lưu lịch sử vào cơ sở dữ liệu và cung cấp API cho dashboard.
9. Dashboard web hiển thị dữ liệu realtime, biểu đồ lịch sử, trạng thái cảm biến, trạng thái điều khiển tự động và các cảnh báo đang kích hoạt.
10. Dashboard cho phép bật/tắt chế độ tự động, điều khiển thủ công từng thiết bị khi ở manual mode và cập nhật ngưỡng điều khiển của hệ thống.

2.1.4 Yêu cầu phi chức năng

Bên cạnh các chức năng chính, hệ thống cần đáp ứng một số yêu cầu phi chức năng để vận hành ổn định trên nền tảng nhúng và phù hợp với mô hình IoT.

Thứ nhất, firmware cần nhẹ và ổn định trên ESP32. Các tác vụ đọc cảm biến, xử lý dữ liệu, điều khiển thiết bị và truyền thông không được làm treo toàn bộ hệ thống khi một cảm biến mất kết nối hoặc phản hồi chậm. Vì vậy, driver cần có timeout, trạng thái lỗi rõ ràng và cơ chế bỏ qua mẫu đo không hợp lệ.

Thứ hai, dữ liệu nội bộ cần được biểu diễn nhất quán và tiết kiệm tài nguyên. Trong firmware hiện tại, một số đại lượng được xử lý nội bộ dưới dạng fixed-point nhân hệ số 10 trước khi chuyển về giá trị phù hợp để publish hoặc hiển thị.

Thứ ba, hệ thống cần có khả năng mở rộng. Việc bổ sung cảm biến mới, thiết bị chấp hành mới hoặc chỉ số IAQ mới không nên làm thay đổi toàn bộ kiến trúc firmware và backend, vì vậy các khối đọc cảm biến, chuẩn hóa mẫu đo, đánh giá IAQ, truyền MQTT và hiển thị dashboard cần được tách tương đối độc lập.

Thứ tư, tầng truyền thông IoT cần có định dạng dữ liệu rõ ràng và dễ kiểm thử. MQTT được sử dụng cho luồng publish/subscribe giữa ESP32 và backend vì đây là giao thức nhẹ, phù hợp với các hệ thống IoT và thiết bị có tài nguyên hạn chế [12]. Payload JSON cần có cấu trúc ổn định, có mã thiết bị, thời gian, giá trị cảm biến và trạng thái điều khiển để backend có thể kiểm tra trước khi lưu trữ.

Thứ năm, backend và dashboard cần hỗ trợ truy vết dữ liệu theo thời gian. Dữ liệu lịch sử cần được lưu kèm timestamp để phục vụ biểu đồ, đối chiếu khi kiểm thử và phân tích xu hướng IAQ. Đồng thời, các cấu hình nhạy cảm như địa chỉ broker hoặc thông tin cơ sở dữ liệu cần được tách khỏi mã nguồn thông qua biến môi trường hoặc tệp cấu hình cục bộ.

Thứ sáu, dashboard cần thể hiện rõ trạng thái cảm biến, trạng thái dữ liệu, trạng thái thiết bị và chế độ điều khiển để người dùng có thể quan sát và thao tác nhất quán.

3. Kiến trúc hệ thống

Kiến trúc tổng thể của hệ thống được tổ chức theo mô hình phân tầng gồm tầng cảm biến, tầng thiết bị nhúng ESP32, tầng truyền thông MQTT, tầng backend/API, tầng cơ sở dữ liệu và tầng dashboard web. Cách tổ chức này giúp phân tách rõ trách nhiệm giữa khâu thu thập dữ liệu, khâu truyền nhận qua mạng, khâu lưu trữ lịch sử và khâu giám sát, điều khiển ở phía người dùng.

3.1 Luồng dữ liệu

Về mặt chức năng, hệ thống gồm hai chiều luồng chính: luồng giám sát từ thiết bị lên máy chủ và luồng điều khiển từ người dùng xuống thiết bị. Luồng giám sát được mô tả như sau:

Sensor -> ESP32/Firmware -> MQTT -> Backend -> Database -> API -> Web
Dashboard

Trong luồng này, cảm biến cung cấp dữ liệu thô cho firmware trên ESP32; firmware thực hiện đọc mẫu, kiểm tra hợp lệ, tổng hợp trạng thái IAQ và publish telemetry qua MQTT. Backend tiếp nhận payload, kiểm tra theo contract chung, cập nhật trạng thái hiện thời, lưu lịch sử vào MySQL và cung cấp API cho dashboard truy xuất.

Ở chiều ngược lại, luồng điều khiển được tổ chức theo hướng:

Web Dashboard -> API -> Backend -> MQTT -> ESP32 -> Device/LED

Theo đó, thao tác của người dùng trên dashboard không tác động trực tiếp đến phần cứng, mà đi qua backend và topic điều khiển MQTT trước khi tới firmware. Cách tổ chức này cho phép chuẩn hóa lệnh điều khiển, lưu vết thao tác và áp dụng các ràng buộc an toàn trước khi thiết bị thực thi.

Trong nguyên mẫu hiện tại, hai chiều luồng trên đã được hiện thực ở mức cơ bản bởi firmware RTOS, backend, cơ sở dữ liệu và dashboard web.

3.2 Luận điểm thiết kế hệ thống

Luận điểm thiết kế cốt lõi của đề tài là ưu tiên xử lý cục bộ tại thiết bị trước, sau đó mới mở rộng sang giám sát và điều khiển từ xa. Điều này có nghĩa là ESP32 vẫn phải duy trì được các chức năng cơ bản như đọc cảm biến, đánh giá IAQ và kích hoạt cảnh báo cục bộ ngay cả khi Wi-Fi, broker MQTT hoặc server tạm thời không khả dụng. Tầng IoT, backend và dashboard vì vậy chỉ đóng vai trò mở rộng khả năng quan sát, lưu trữ lịch sử và tương tác từ xa.

Từ luận điểm này, các quyết định thiết kế được sắp xếp theo thứ tự ưu tiên sau: dữ liệu cảm biến phải được kiểm tra trước khi dùng cho điều khiển; cảnh báo cục bộ phải có khả năng vận hành độc lập với server; các lệnh từ dashboard phải đi qua backend và contract truyền thông thống nhất; còn cơ sở dữ liệu phục vụ chủ yếu cho lưu trữ và phân tích lịch sử thay vì là điều kiện bắt buộc để thiết bị phản ứng tức thời.

3.3 Phân rã module

Trên cơ sở kiến trúc phân tầng, hệ thống được phân rã thành các module với trách nhiệm xử lý tương đối độc lập. Việc phân rã này nhằm làm rõ ranh giới chức năng giữa các khối thành phần và quan hệ trao đổi dữ liệu giữa chúng.

Bảng 3.1: Phân rã module theo trách nhiệm kiến trúc

Module	Chức năng chính	Quan hệ với các module khác
<code>rtos</code>	Đọc cảm biến, xử lý dữ liệu cục bộ, đánh giá IAQ, điều khiển thiết bị và giao tiếp MQTT ở phía ESP32	Nhận cấu hình và lệnh điều khiển từ backend qua MQTT; gửi telemetry và trạng thái thiết bị lên các tầng phía trên
<code>common</code>	Định nghĩa topic MQTT, cấu trúc payload JSON và contract dữ liệu dùng chung	Là lớp giao ước giữa firmware, backend, cơ sở dữ liệu và frontend
<code>iot_backend/backend</code>	Tiếp nhận dữ liệu MQTT, kiểm tra payload, duy trì trạng thái hiện thời, cung cấp API và phát lệnh điều khiển	Kết nối giữa <code>rtos</code> , <code>database</code> và <code>frontend</code> ; là điểm trung gian cho cả giám sát lẫn điều khiển
<code>iot_backend/database</code>	Lưu trữ telemetry lịch sử và nhật ký lệnh điều khiển	Nhận dữ liệu ghi từ backend và cung cấp nguồn dữ liệu lịch sử cho API/dashboard
<code>iot_backend/frontend</code>	Hiển thị dữ liệu realtime, dữ liệu lịch sử, trạng thái thiết bị và giao diện điều khiển người dùng	Khai thác dữ liệu từ backend qua API; không giao tiếp trực tiếp với phần cứng
<code>integration_test</code>	Hỗ trợ kiểm thử tích hợp luồng dữ liệu và luồng điều khiển ở mức hệ thống	Kiểm chứng sự tương thích giữa contract, backend và các luồng MQTT trước khi kiểm thử end-to-end thực tế

3.4 Nguyên tắc thiết kế

Ở mức triển khai, các tầng và module được tách theo trách nhiệm xử lý. Nhóm driver chỉ đảm nhiệm giao tiếp với cảm biến và kiểm tra trạng thái giao thức. Lớp gom mẫu dữ liệu chuẩn hóa thông tin đo về một cấu trúc chung để phục vụ các bước xử lý tiếp theo. Khối đánh giá IAQ chỉ tập trung vào logic suy diễn trạng thái môi trường và ngưỡng điều khiển, không phụ thuộc trực tiếp vào phần cứng đầu ra. Khối điều khiển đầu ra chịu trách nhiệm áp dụng trạng thái thiết bị theo kết quả đánh giá hoặc theo lệnh điều khiển hợp lệ.

Trong phiên bản RTOS, các thành phần này được tách thành nhiều tác vụ và trao đổi dữ liệu thông qua queue, mutex và semaphore. Ở phía server, backend và dashboard

không làm việc trực tiếp với phần cứng mà giao tiếp với thiết bị thông qua MQTT và API dựa trên cùng một contract dữ liệu. Cách tổ chức này giúp hệ thống nhất quán giữa tầng nhúng và tầng ứng dụng, đồng thời thuận lợi cho việc mở rộng về sau.

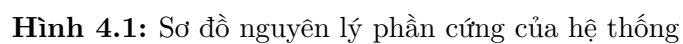
4. Thiết kế phần cứng

4.1 Thiết kế phần cứng

4.1.1 Nền tảng điều khiển

Hệ thống sử dụng ESP32 NodeMCU-32S làm nền tảng điều khiển trung tâm. ESP32 được lựa chọn vì tích hợp kết nối Wi-Fi, có nhiều giao tiếp ngoại vi như UART, ADC và GPIO, đồng thời hỗ trợ tốt cho việc tổ chức firmware theo mô hình đa tác vụ. Với định hướng của đề tài, ESP32 đóng vai trò nút biên thực hiện ba nhóm chức năng chính: thu thập dữ liệu từ các cảm biến môi trường, xử lý và phản ứng cục bộ, đồng thời truyền dữ liệu lên backend để phục vụ giám sát và điều khiển từ xa [11].

Trong cấu hình hiện tại, firmware sử dụng một UART cho cảm biến bụi PMS7003, một UART khác cho bus RS485/Modbus và một kênh ADC cho cảm biến VOC dạng analog. Cách phân bố này phù hợp với đặc tính tín hiệu của từng cảm biến, đồng thời tận dụng được các tài nguyên phần cứng sẵn có trên ESP32 mà không cần thêm vi điều khiển phụ trợ.



Bảng 4.1: Pin map chính trong firmware

Chức năng	Pin ESP32	Ghi chú
PMS RX	GPIO 26	ESP32 nhận dữ liệu từ chân TX của PMS7003.
PMS TX	GPIO 27	Dự phòng khi cần gửi lệnh cấu hình cho PMS7003.
RS485 RX	GPIO 16	UART2 nhận dữ liệu phản hồi từ module RS485.
RS485 TX	GPIO 17	UART2 gửi request Modbus RTU đến các cảm biến RS485.
RS485 DE/RE	-1	Source giả định module RS485 tự điều hướng truyền/nhận, firmware không điều khiển chân riêng.
VOC ADC	GPIO 34	Chân ADC1 input-only, dùng để đọc tín hiệu analog từ cảm biến VOC.

4.1.2 Khối nguồn

Hệ thống sử dụng nguồn một chiều đầu vào 21.5V làm nguồn cấp chính cho toàn mạch. Ở đầu vào nguồn, cầu chì 1A được sử dụng để bảo vệ mạch khi xảy ra sự cố quá dòng hoặc chập chập. Diode Schottky 1N5819 được đặt nối tiếp nhằm bảo vệ mạch trong trường hợp đấu ngược cực nguồn, đồng thời giảm tổn hao điện áp so với các diode chỉnh lưu thông thường. Sau khối bảo vệ, nguồn được đưa qua công tắc ON/OFF để tạo nhánh nguồn đóng cắt chính cho hệ thống.

Từ nhánh nguồn 21.5V sau công tắc, một hướng được cấp cho các cảm biến sử dụng mức áp cao hơn, đồng thời một hướng được đưa vào bộ hạ áp LM2596 để tạo ra đường 5V cấp cho ESP32 và các khối logic liên quan. Cách tổ chức này giúp tách rõ nhánh nguồn đầu vào và nhánh nguồn hạ áp, đồng thời thuận lợi cho việc mở rộng phần cứng ở các phiên bản sau.

Tại nhánh 21.5V sau công tắc và tại đường 5V sau bộ hạ áp, mạch sử dụng kết hợp tụ điện phân và tụ gốm mắc song song xuống mass. Tụ điện phân 100 μF ở phía đầu vào và 470 μF ở phía 5V đóng vai trò tụ đệm năng lượng, giúp giảm sụt áp tức thời khi tải thay đổi. Tụ gốm 0.1 μF được dùng để triệt thành phần nhiễu cao tần trên đường nguồn, từ đó cải thiện độ ổn định điện áp cấp cho vi điều khiển, module RS485 và các cảm biến.

4.1.3 Cảm biến và đặc tính kết nối

Các cảm biến được lựa chọn nhằm theo dõi những đại lượng có ảnh hưởng trực tiếp đến chất lượng không khí trong phòng học, bao gồm bụi mịn, CO_2 , nhiệt độ, độ ẩm và VOC. Mỗi loại cảm biến sử dụng một phương thức giao tiếp phù hợp với đặc tính tín hiệu và yêu cầu truyền dữ liệu.

Cảm biến bụi PMS7003 được dùng để đo các chỉ số $\text{PM}_{1.0}$, $\text{PM}_{2.5}$ và PM_{10} . Cảm

biến này làm việc theo cơ chế xuất frame nhị phân qua UART ở tốc độ 9600 bps; trong firmware, driver tìm cặp byte header `0x42 0x4D`, đọc đủ frame, kiểm tra checksum và sau đó tách ra các giá trị bụi cần sử dụng [22]. Cách giao tiếp này đơn giản và phù hợp với yêu cầu đọc dữ liệu bụi theo chu kỳ trên ESP32.

Cảm biến CO_2 và module nhiệt độ/độ ẩm được kết nối qua chuẩn RS485 Modbus RTU. Firmware tạo request với function code `0x03`, tính CRC16, gửi qua UART2 và kiểm tra response trước khi cập nhật dữ liệu. Trong cấu hình hiện tại, cảm biến CO_2 sử dụng slave ID 1 và đọc register `0x0001`; module nhiệt độ/độ ẩm sử dụng slave ID 2 và đọc hai register bắt đầu từ `0x0000`. Việc sử dụng RS485 giúp kết nối cảm biến theo hướng chống nhiễu tốt hơn và thuận lợi hơn cho mở rộng bus cảm biến về sau [21].

Cảm biến VOC được đọc bằng ngõ vào ADC của ESP32. Do tín hiệu ở dạng analog và dễ dao động theo nhiễu tức thời, lớp `VocAnalog` cấu hình attenuation cho ADC và sử dụng bộ đệm vòng để tính trung bình trượt. Cách xử lý này giúp làm mượt dữ liệu trước khi đưa vào tầng đánh giá IAQ, phù hợp với vai trò của VOC như một chỉ số xu hướng trong nguyên mẫu hiện tại.

4.1.4 Lý do lựa chọn giao tiếp cảm biến

Việc lựa chọn giao tiếp cho từng cảm biến dựa trên đặc tính dữ liệu và mức độ ổn định cần thiết của đường truyền. PMS7003 sử dụng UART vì cảm biến xuất dữ liệu dạng frame liên tục, có header và checksum nên dễ kiểm tra tính hợp lệ. Đối với cảm biến CO_2 và nhiệt độ/độ ẩm, RS485 Modbus RTU phù hợp hơn do hỗ trợ truyền có dây ổn định, có địa chỉ slave và có CRC để phát hiện lỗi truyền. Trong khi đó, VOC dùng ngõ vào ADC vì cảm biến xuất tín hiệu analog và không yêu cầu một giao thức truyền số riêng.

Cách phân chia này giúp hệ thống tách rõ ba nhóm giao tiếp: UART frame cho cảm biến bụi, Modbus register cho cảm biến RS485 và analog sampling cho cảm biến VOC. Nhờ đó, tầng `SensorHub` có thể gom dữ liệu từ các nguồn khác nhau thành cùng một mẫu đo thống nhất, trong khi các tầng xử lý phía sau không cần phụ thuộc trực tiếp vào chi tiết giao thức của từng cảm biến.

4.1.5 Thiết bị chấp hành và cảnh báo

Trong phạm vi nguyên mẫu, hệ thống sử dụng LED để mô phỏng thiết bị chấp hành và cảnh báo. Nhóm LED xanh biểu diễn trạng thái của các thiết bị hỗ trợ cải thiện chất lượng không khí như thông gió, lọc HEPA, lọc than hoạt tính, điều hòa và tạo ẩm. Nhóm LED đỏ biểu diễn cảnh báo theo từng nguyên nhân như CO_2 , bụi mịn, VOC, nhiệt độ và độ ẩm.

Cách sử dụng LED giúp việc kiểm thử hệ thống trở nên trực quan hơn trong giai đoạn phát triển mà không cần kết nối trực tiếp với các thiết bị công suất lớn. Khi một chỉ số môi trường vượt ngưỡng, firmware có thể bật LED cảnh báo tương ứng; khi hệ thống

phát lệnh điều khiển thiết bị, LED xanh tương ứng sẽ thay đổi trạng thái để mô phỏng quá trình bật hoặc tắt thiết bị. Mỗi LED được mắc nối tiếp với điện trở hạn dòng $120\ \Omega$ nhằm giới hạn dòng qua LED, bảo vệ chân GPIO của ESP32 và duy trì độ sáng ở mức phù hợp cho quan sát. Các chân điều khiển LED được khai báo trong `Pins.h` và có thể thay đổi theo wiring thực tế.

5. Firmware đọc dữ liệu và điều khiển cục bộ

5.1 Firmware đọc dữ liệu và xử lý cục bộ

5.1.1 Vai trò của firmware

Firmware chính của hệ thống nằm trong nhánh `rtos` và chạy trên ESP32 bằng PlatformIO/Arduino framework kết hợp FreeRTOS. Vai trò của firmware là thu thập dữ liệu từ các cảm biến, chuẩn hóa thành mẫu đo thống nhất, đánh giá trạng thái chất lượng không khí và điều khiển cảnh báo cục bộ trước khi dữ liệu được gửi lên backend. Các sketch kiểm thử phần cứng chỉ phục vụ giai đoạn thử nghiệm từng cảm biến riêng lẻ, không phải luồng vận hành chính của hệ thống.

Xét theo chức năng, firmware có thể được chia thành ba lớp. Lớp thứ nhất là các driver giao tiếp với PMS7003, RS485 Modbus và ADC. Lớp thứ hai là `SensorHub`, có nhiệm vụ gom dữ liệu từ nhiều nguồn thành một mẫu đo thống nhất. Lớp thứ ba là khối IAQ, gồm `IaqEvaluator` và `IaqController`, chịu trách nhiệm suy diễn trạng thái môi trường và ánh xạ kết quả sang thiết bị mô phỏng và cảnh báo.

Sensor drivers -> SensorHub -> SensorSample -> IaqEvaluator ->
IaqController/MQTT

5.1.2 Mô hình dữ liệu trung tâm

Đơn vị dữ liệu trung tâm của firmware là `SensorSample`. Cấu trúc này giữ các giá trị đo dưới dạng phù hợp cho xử lý nhúng, trong đó nhiệt độ, độ ẩm và một số đại lượng nội bộ được biểu diễn theo fixed-point nhân hệ số 10 để giảm phụ thuộc vào tính toán số thực ở tầng đọc cảm biến. Đồng thời, mỗi nhóm dữ liệu đều có cờ hợp lệ hoặc giá trị sentinel đi kèm để phân biệt giữa dữ liệu hợp lệ và dữ liệu thiếu.

Bảng 5.1: Các nhóm trường chính của **SensorSample**

Nhóm trường	Vai trò trong firmware
Thời gian đo	Lưu mốc thời gian tương đối tại thời điểm đọc mẫu
PM1.0/PM2.5/PM10	Giữ dữ liệu bụi mịn từ PMS7003; dùng PM2.5 làm chỉ số chính cho pipeline IoT
Nhiệt độ, độ ẩm	Lưu giá trị từ cảm biến RS485 dưới dạng fixed-point x10
CO₂	Lưu nồng độ CO ₂ theo ppm
VOC	Lưu giá trị VOC raw và giá trị trung bình trượt dùng cho đánh giá
Cờ hợp lệ	Cho biết từng nhóm cảm biến có đọc thành công hay không

Cách tổ chức này cho phép tầng đánh giá IAQ làm việc trên một cấu trúc dữ liệu đồng nhất, thay vì phải phụ thuộc vào chi tiết giao thức của từng cảm biến. Đây cũng là cơ sở để ánh xạ dữ liệu sang **SensorData_t** và telemetry JSON ở các bước sau.

5.1.3 Đọc cảm biến và chuẩn hóa dữ liệu

SensorHub là điểm gom dữ liệu từ ba nhóm giao tiếp phần cứng. Khi được khởi tạo, thành phần này cấu hình UART1 cho PMS7003, UART2 cho RS485 Modbus và ADC1 cho cảm biến VOC. Trong quá trình đọc mẫu, từng cảm biến được truy xuất với timeout riêng; nếu một cảm biến lỗi, mẫu đo vẫn được trả về nhưng nhóm dữ liệu tương ứng được đánh dấu không hợp lệ.

Bảng 5.2: Các driver cảm biến và cách xử lý dữ liệu

Cảm biến	Giao tiếp	Cách xử lý chính
PMS7003	UART1 9600 bps	Tìm header 0x42 0x4D, đọc đủ frame, kiểm tra checksum và tách PM1.0/PM2.5/PM10
CO₂ RS485	Modbus RTU	Gửi request function 0x03, đọc register 0x0001, kiểm tra độ dài response, slave ID, function code và CRC16
Nhiệt độ/độ ẩm RS485	Modbus RTU	Gửi request function 0x03, đọc hai register từ 0x0000, chuyển dữ liệu về dạng fixed-point x10
VOC analog	ADC1 GPIO34	Đọc ADC 12-bit, cấu hình attenuation và dùng trung bình trượt để giảm nhiễu tức thời

Hai nguyên tắc quan trọng của tầng driver là timeout và kiểm tra giao thức. PMS7003 chỉ được xem là hợp lệ khi checksum đúng. Với Modbus, response chỉ được chấp nhận khi khớp slave ID, function code, byte count và CRC16. Nhờ đó, lỗi dây nối, cảm biến chưa cấp nguồn hoặc nhiều đường truyền không dẫn đến việc firmware dùng nhầm giá trị rác cho tầng đánh giá.

5.1.4 Đánh giá IAQ và cơ chế hysteresis

IaqEvaluator là thành phần chuyển SensorSample thành IaqState. Mỗi đại lượng được so sánh với một cặp ngưỡng bật/tắt để quyết định có cần kích hoạt thiết bị xử lý hay không. Cơ chế này được triển khai theo hướng hysteresis, nghĩa là hệ thống không dùng một ngưỡng duy nhất cho cả quá trình bật và tắt thiết bị. Thay vào đó, thiết bị chỉ được bật khi giá trị đo vượt ngưỡng trên và chỉ tắt khi giá trị quay trở lại dưới ngưỡng hồi phục.

Bảng 5.3: Các ngưỡng điều khiển mặc định trong firmware

Đại lượng	Cặp ngưỡng mặc định	Ý nghĩa vận hành
CO ₂	1000/950 ppm	Bật xử lý khi CO ₂ cao và chỉ tắt khi đã giảm xuống dưới ngưỡng hồi phục
PM2.5	35/30 $\mu\text{g}/\text{m}^3$	Điều khiển lọc bụi theo mức bụi mịn
VOC	500/450	Điều khiển thông gió và lọc than theo VOC raw/index
Nhiệt độ	32.0/31.0 °C	Điều khiển thiết bị làm mát mô phỏng
Độ ẩm thấp	30.0/35.0%RH	Bật/tắt tạo ẩm
Độ ẩm cao	65.0/60.0%RH	Bật/tắt thông gió khi quá ẩm

Ngoài hysteresis, firmware còn áp dụng hold-time mặc định 5000 ms cho các đầu ra xử lý. Khi trạng thái vừa đổi, thiết bị được giữ tối thiểu trong một khoảng thời gian trước khi được phép đổi tiếp. Cơ chế này giúp giảm hiện tượng bật/tắt liên tục khi dữ liệu dao động gần ngưỡng, đặc biệt với các tín hiệu có nhiễu hoặc biến động ngắn hạn.

5.1.5 Bảng nhân quả vận hành hệ thống

Từ góc nhìn vận hành, mỗi nhóm đại lượng môi trường sẽ dẫn đến một tập hành vi điều khiển tương ứng. Bảng dưới đây tóm tắt quan hệ nhân quả giữa nguyên nhân môi trường và đầu ra xử lý trong IaqState.

Bảng 5.4: Bảng nhân quả vận hành của hệ thống

Nguyên nhân	Điều kiện suy diễn	Phản ứng của hệ thống
CO ₂ cao	Nồng độ CO ₂ vượt ngưỡng cấu hình	Bật <code>wantVent</code> , bật <code>wantCarbon</code> , kích hoạt <code>alarmCO2</code>
PM2.5 cao	PM2.5 vượt ngưỡng bụi mịn	Bật <code>wantHepa</code> , kích hoạt <code>alarmPM</code>
VOC cao	VOC raw/index vượt ngưỡng cấu hình	Bật <code>wantVent</code> , có thể bật <code>wantCarbon</code> , kích hoạt <code>alarmVOC</code>
Nhiệt độ cao	Nhiệt độ vượt ngưỡng trên	Bật <code>wantAc</code> , kích hoạt <code>alarmTemp</code>
Độ ẩm thấp	Độ ẩm dưới ngưỡng thấp	Bật <code>wantHumid</code> , kích hoạt <code>alarmRH</code> khi vượt vùng cảnh báo
Độ ẩm cao	Độ ẩm vượt ngưỡng cao	Bật <code>wantVent</code> , kích hoạt <code>alarmRH</code> khi vượt vùng cảnh báo
Dữ liệu thiếu/lỗi	Cảm biến timeout, mất kết nối hoặc sai giao thức	Giữ cờ trạng thái lỗi, tránh dùng dữ liệu không hợp lệ cho quyết định điều khiển

5.1.6 Áp dụng điều khiển và lưu dữ liệu

IaqController không chứa thuật toán đánh giá chất lượng không khí. Thành phần này chỉ nhận IaqState và ghi trạng thái ra các chân LED mô phỏng thiết bị và cảnh báo. Nhờ sự tách biệt này, về sau có thể thay LED bằng relay hoặc module công suất mà không cần thay đổi thuật toán IAQ.

Trong luồng IoT chính, telemetry không được lưu cục bộ dưới dạng CSV trên firmware RTOS. Thay vào đó, ESP32 publish dữ liệu lên topic `iot/sensor/data`; backend tiếp nhận, chuẩn hóa giá trị thiếu và lưu vào bảng `sensor_data`. Vì vậy, dữ liệu ở MySQL là nguồn chính để đối chiếu kết quả đo, kiểm tra lịch sử và hiển thị biểu đồ trên dashboard.

6. Thiết kế RTOS và IoT truyền dữ liệu

6.1 Thiết kế RTOS và tích hợp IoT

6.1.1 Mục tiêu tổ chức RTOS

Phiên bản `rtos` tổ chức firmware thành nhiều tác vụ chạy song song trên ESP32. Mục tiêu của cách tổ chức này là tách các công việc có đặc tính thời gian khác nhau, bao gồm đọc cảm biến định kỳ, xử lý IAQ khi có mẫu mới, cập nhật cảnh báo, duy trì Wi-Fi/MQTT và xử lý lệnh điều khiển từ dashboard. Nhờ vậy, thao tác mạng hoặc việc broker mất kết nối không làm dừng phần đọc cảm biến và phản ứng cục bộ.

Firmware RTOS được build bằng PlatformIO với board `nodemcu-32s`, Arduino framework, `PubSubClient` cho MQTT và `ArduinoJson` cho payload. Các cấu hình nhảy cảm như Wi-Fi, MQTT broker và `DEVICE_ID` được tách trong `include/local_config.h`, giúp thuận lợi cho triển khai mà không làm thay đổi mã nguồn lõi.

6.1.2 Khởi tạo hệ thống

Trong `rtos/src/main.cpp`, quá trình khởi tạo gồm bốn bước chính. Trước hết, firmware khởi tạo `SensorHub` với các giao tiếp phần cứng cần thiết. Tiếp theo, hệ thống nạp cấu hình bền vững từ NVS, bao gồm trạng thái auto-control và `ThresholdConfig`. Sau đó, `IaqEvaluator` và `IaqController` được khởi tạo theo cấu hình hiện hành. Cuối cùng, các tài nguyên FreeRTOS như queue, mutex và semaphore được tạo, rồi các task chính được đưa vào vận hành.

Listing 6.1: Các task chính trong phiên bản RTOS

```
xTaskCreatePinnedToCore(vTaskAlert,          "Task Alert",    2048, NULL, 5, NULL, 1);
xTaskCreatePinnedToCore(vTaskSensorRead,     "Task Sensor",   4096, NULL, 4, NULL, 1);
xTaskCreatePinnedToCore(vTaskDataProcess,    "Task Process",  4096, NULL, 3, NULL, 1);
xTaskCreatePinnedToCore(vTaskControl,        "Task Control",  8192, NULL, 3, NULL, 1);
xTaskCreatePinnedToCore(vTaskMQTT,           "Task MQTT",     8192, NULL, 1, NULL, 0);
```

Trong cấu hình hiện tại, các tác vụ đọc cảm biến, xử lý và điều khiển được đặt trên

core 1, còn task MQTT được đặt trên core 0 với mức ưu tiên thấp hơn. Cách phân bố này phù hợp với nguyên tắc ưu tiên phản ứng cục bộ: các thao tác mạng có thể retry kết nối mà không cạnh tranh trực tiếp với đường xử lý IAQ.

6.1.3 Tài nguyên đồng bộ

Các tài nguyên FreeRTOS được tạo tập trung trong `global.cpp`. Dữ liệu cảm biến được truyền qua queue; trạng thái hiện thời được bảo vệ bằng mutex; còn các thay đổi cảnh báo có thể đánh thức task xử lý đầu ra thông qua semaphore.

Bảng 6.1: Các tài nguyên RTOS chính

Tài nguyên	Cơ chế	Vai trò
<code>xSensorQueue</code>	Queue	Truyền dữ liệu đã chuẩn hóa từ task đọc cảm biến sang task xử lý
<code>xControlQueue</code>	Queue	Truyền payload điều khiển MQTT từ callback mạng sang task điều khiển
<code>xDataMutex</code>	Mutex	Bảo vệ <code>g_LatestData</code> , <code>g_iaq</code> và trạng thái run-time khi nhiều task cùng truy cập
<code>xAlertSem</code>	Counting semaphore	Báo cho task cảnh báo cập nhật đầu ra khi trạng thái IAQ thay đổi
<code>SnapshotStore</code>	Critical section	Lưu mẫu <code>SensorSample</code> mới nhất ở tầng firmware

Việc sử dụng queue và mutex giúp các khối chức năng trao đổi dữ liệu theo cách tường minh, giảm phụ thuộc trực tiếp giữa các task. Đồng thời, callback MQTT được giữ ngắn gọn bằng cách chỉ đưa payload vào queue thay vì xử lý JSON trực tiếp trong ngữ cảnh mạng.

6.1.4 Phân rã task trong firmware

Bảng 6.2: Các task RTOS trong firmware

Task	Chu kỳ/sự kiện	Ưu tiên	Chức năng chính
vTaskAlert	Semaphore hoặc 500 ms	5	Đọc g_iaq và áp dụng trạng thái ra LED/thiết bị mô phỏng
vTaskSensorRead	2000 ms	4	Đọc SensorHub, cập nhật SnapshotStore, ánh xạ sang SensorData_t và gửi vào queue
vTaskDataProcess	Khi queue có dữ liệu	3	Gọi IaqEvaluator, tính alert_level, cập nhật trạng thái dùng chung
vTaskControl	Khi có command	3	Xử lý auto/manual mode, command thiết bị, threshold và lưu NVS
vTaskMQTT	Định kỳ và callback MQTT	1	Duy trì Wi-Fi/MQTT, publish telemetry và subscribe control topic

Trong các task này, `vTaskDataProcess` là điểm nối giữa dữ liệu cảm biến và quyết định điều khiển. Task này tính `alert_level` từ `IaqState`, đồng thời giữ lại trạng thái thiết bị hiện thời nếu hệ thống đang ở manual mode. Nhờ đó, hệ thống vẫn có thể tiếp tục đánh giá cảnh báo mà không ghi đè quyết định điều khiển thủ công của người dùng.

6.1.5 Luồng telemetry từ ESP32 lên backend

Sau khi dữ liệu được xử lý, firmware ánh xạ từ `SensorSample` sang `SensorData_t` và sau đó sang payload JSON để publish qua MQTT. Các quy ước dữ liệu thiếu được giữ thống nhất với contract chung: nhiệt độ và độ ẩm lỗi được đưa về NAN, PM2.5 lỗi thành -1, CO₂ lỗi thành 0xFFFF. Timestamp được tạo bằng `esp_timer_get_time()` để biểu diễn thời gian tương đối kể từ lúc thiết bị khởi động.

Bảng 6.3: Các nhóm trường telemetry chính

Nhóm field	Ý nghĩa
device_id, timestamp, uptime_ms	Định danh thiết bị và mốc thời gian đo
temperature, humidity, pm25, co2, voc	Dữ liệu đo đã chuẩn hóa
alert_level	Mức cảnh báo tổng hợp của hệ thống
sensor_health	Trạng thái từng nhóm cảm biến
alerts	Cờ cảnh báo theo từng nguyên nhân
devices	Trạng thái các thiết bị xử lý đang được yêu cầu bật
auto_control_enabled, config_version	Trạng thái chế độ điều khiển và phiên bản cấu hình

Backend đọc cùng contract dữ liệu qua `common/data_format.json`, nhờ đó firmware, MQTT service, database và dashboard cùng dùng một cấu trúc payload thống nhất. Điều này giúp giảm sai lệch giữa các tầng và đơn giản hóa việc kiểm thử tích hợp.

6.1.6 MQTT contract và luồng điều khiển

Trong hệ thống hiện tại, ba topic MQTT chính được sử dụng cho telemetry, trạng thái thiết bị và lệnh điều khiển.

Bảng 6.4: Các topic MQTT chính

Topic	Hướng	Vai trò
iot/sensor/data	ESP32 → backend	Telemetry định kỳ từ firmware
iot/device/status	ESP32 → backend	ACK hoặc trạng thái sau khi xử lý command
iot/device/control	Backend → ESP32	Lệnh điều khiển từ API/dashboard

Ở chiều điều khiển, dashboard gọi REST API, backend kiểm tra request rồi publish payload JSON xuống `iot/device/control`. Callback MQTT trên ESP32 chỉ sao chép payload vào `xControlQueue`; việc phân tích JSON và thực thi lệnh được chuyển sang `vTaskControl`. Cách tổ chức này giúp tránh làm callback mạng trở nên phức tạp và giữ rõ ranh giới giữa tầng truyền thông với tầng điều khiển.

Bảng 6.5: Nhóm lệnh điều khiển được firmware hỗ trợ

Lệnh	Vai trò
GET_STATUS, GET_AUTO	Trả về trạng thái runtime hiện tại
SET_AUTO	Bật/tắt chế độ auto-control và lưu vào NVS
SET_DEVICE	Điều khiển từng thiết bị khi firmware đang ở manual mode
GET_THRESHOLDS	Publish cấu hình ngưỡng hiện tại lên status topic
SET_THRESHOLDS	Cập nhật ngưỡng, kiểm tra hysteresis, lưu NVS và tăng <code>config_version</code>
RESET_THRESHOLDS	Khôi phục ngưỡng mặc định
REBOOT	Publish ACK rồi khởi động lại ESP32

Một chính sách an toàn quan trọng được đặt ở phía firmware: khi `auto_control_enabled` đang bật, các lệnh manual điều khiển thiết bị sẽ bị từ chối. Cơ chế này tránh việc dashboard vô tình ghi đè quyết định xử lý cục bộ đang được thuật toán IAQ yêu cầu kích hoạt.

6.1.7 Cấu hình ngưỡng và vận hành khi mất kết nối

Ngưỡng điều khiển được lưu trong `ThresholdConfig`. Khi nhận lệnh `SET_THRESHOLDS`, firmware tạo cấu hình tạm, kiểm tra quan hệ hysteresis, cập nhật `IaqEvaluator` rồi lưu xuống NVS. Nếu lưu thất bại, hệ thống rollback về cấu hình trước đó. Cách làm này giúp các thiết lập từ dashboard không bị mất sau khi ESP32 khởi động lại.

Thiết kế hiện tại cũng không phụ thuộc hoàn toàn vào server. Nếu Wi-Fi hoặc MQTT mất kết nối, các task đọc cảm biến, xử lý IAQ, cảnh báo và điều khiển cục bộ vẫn tiếp tục chạy. `vTaskMQTT` chỉ đảm nhiệm việc định kỳ thử kết nối lại và tiếp tục publish khi hạ tầng mạng phục hồi. Giới hạn hiện tại là firmware chưa có hàng đợi lưu lịch sử offline để đẩy bù dữ liệu cũ; dữ liệu lịch sử chính vẫn được lưu ở MySQL sau khi backend nhận được MQTT.

7. Server, cơ sở dữ liệu và dashboard web

7.1 Backend server

Backend của hệ thống được xây dựng bằng Node.js và Express, đóng vai trò lớp trung gian giữa broker MQTT, cơ sở dữ liệu MySQL và dashboard web. Nhiệm vụ chính của backend là tiếp nhận telemetry từ ESP32, chuẩn hóa payload theo contract chung, duy trì trạng thái mới nhất trong bộ nhớ, lưu dữ liệu lịch sử xuống cơ sở dữ liệu và cung cấp REST API cho phía giao diện người dùng.

MQTT service -> payload mapper -> latest cache -> database -> REST API

Về mặt triển khai, backend tách các chức năng thành các nhóm tương đối độc lập: lớp MQTT service đảm nhiệm subscribe/publish với broker; lớp controller xử lý logic nghiệp vụ; lớp model thao tác với MySQL; còn hệ thống route công bố API cho dashboard. Cách tổ chức này giúp luồng đo lường thời gian thực và luồng điều khiển từ web có thể dùng chung contract dữ liệu nhưng vẫn tách biệt về trách nhiệm.

Bảng 7.1: Các API chính của backend

API	Chức năng	Nguồn hoặc đích dữ liệu
GET /api/data	Trả về trạng thái realtime mới nhất của thiết bị	RAM/cache backend
GET /api/history	Truy vấn dữ liệu lịch sử theo khoảng thời gian	MySQL
GET /api/export.csv	Xuất dữ liệu lịch sử dạng CSV phục vụ phân tích ngoài hệ thống	MySQL
GET /api/stats	Tính thống kê nhanh theo cửa sổ thời gian	MySQL
POST /api/control	Gửi nhóm lệnh legacy hoặc lệnh kiểm thử	MQTT control topic
POST /api/control/auto	Bật hoặc tắt chế độ auto-control	MQTT control topic
POST /api/control/device	Điều khiển từng thiết bị trong manual mode	MQTT control topic
GET/POST /api/control/thresholds	Yêu cầu hoặc cập nhật ngưỡng điều khiển	MQTT status/control topic
GET /api/control/history	Lấy lịch sử lệnh điều khiển đã gửi	MySQL
GET /health, /health/live, /health/ready	Kiểm tra trạng thái vận hành của backend, database và MQTT	Runtime service

7.1.1 Thiết kế lưu trữ dữ liệu

Tầng lưu trữ sử dụng MySQL với hai bảng chính là `sensor_data` và `control_log`. Bảng `sensor_data` lưu lịch sử telemetry đã được backend chuẩn hóa từ payload MQTT. Bảng `control_log` lưu dấu vết của các lệnh mà backend đã publish xuống thiết bị, từ đó hỗ trợ truy vết hoạt động điều khiển.

Bảng 7.2: Các bảng dữ liệu chính trong backend

Bảng	Cột chính	Vai trò
<code>sensor_data</code>	<code>device_id</code> , <code>temperature</code> , <code>humidity</code> , <code>pm25</code> , <code>co2</code> , <code>voc</code> , <code>alert_level</code> , <code>timestamp</code> , <code>created_at</code>	Lưu chuỗi telemetry theo thời gian
<code>control_log</code>	<code>device_id</code> , <code>command</code> , <code>status</code> , <code>created_at</code>	Lưu lịch sử lệnh điều khiển và trạng thái publish

Trong thiết kế hiện tại, các cột `device_id`, `timestamp` và `created_at` được đánh chỉ mục để phục vụ truy vấn theo thiết bị và theo khoảng thời gian. Việc tách lịch sử telemetry và lịch sử điều khiển thành hai bảng riêng giúp giữ mô hình dữ liệu đơn giản, đồng thời phù hợp với hai nhu cầu phân tích khác nhau: phân tích điều kiện môi trường và truy vết thao tác điều khiển.

7.1.2 Mô hình dữ liệu realtime và lịch sử

Backend tách rõ hai lớp dữ liệu: dữ liệu realtime và dữ liệu lịch sử. Dữ liệu realtime là mẫu mới nhất nhận được từ MQTT và được duy trì trong RAM để API `/api/data` trả về nhanh cho dashboard. Dữ liệu lịch sử được ghi xuống MySQL và được truy xuất qua `/api/history`, `/api/stats` hoặc `/api/export.csv`. Cách tổ chức này giảm phụ thuộc của giao diện realtime vào tốc độ truy vấn cơ sở dữ liệu, trong khi vẫn bảo đảm khả năng lưu trữ và phân tích dài hạn.

Bảng 7.3: Các mốc thời gian được sử dụng trong hệ thống

Trường	Nguồn tạo	Ý nghĩa
timestamp / timestamp_ms	ESP32	Mốc thời gian tương đối của mẫu đo kể từ lúc thiết bị khởi động
uptime_ms	ESP32	Thời gian hoạt động của thiết bị theo millisecond
received_at	Backend	Thời điểm backend nhận được payload mới nhất từ MQTT
created_at	MySQL	Thời điểm record được ghi vào cơ sở dữ liệu

`received_at` chỉ được dùng ở tầng runtime để xác định thiết bị còn online hay không; trường này không được lưu trong bảng `sensor_data`. Backend xem thiết bị là offline nếu quá một khoảng timeout mà không nhận được telemetry mới. Cách tiếp cận này cho phép dashboard phân biệt được hai trạng thái: thiết bị không gửi thêm dữ liệu và thiết bị vẫn online nhưng có một số cảm biến riêng lẻ bị thiếu dữ liệu.

7.1.3 Dashboard web

Frontend của hệ thống được triển khai dưới dạng dashboard tĩnh bằng HTML, CSS và JavaScript, kết hợp Bootstrap cho bố cục giao diện và Chart.js cho trực quan hóa dữ liệu. Về chức năng, giao diện được chia thành ba màn hình chính: dashboard realtime, analytics/history và control panel.

Bảng 7.4: Các màn hình chính của dashboard web

Màn hình	Dữ liệu chính	Vai trò
Realtime dashboard	/api/data	Hiển thị giá trị CO ₂ , PM2.5, VOC, nhiệt độ, độ ẩm, trạng thái kết nối, sensor health và các cờ cảnh báo
Analytics/history	/api/history, /api/stats, /api/export.csv	Hiển thị biểu đồ, thống kê và hỗ trợ xuất dữ liệu CSV
Control panel	/api/control*	Điều khiển auto/manual, bật tắt từng thiết bị và cập nhật ngưỡng threshold

Ở màn hình realtime, giao diện tập trung vào trạng thái môi trường hiện tại và chất lượng dữ liệu cảm biến. Ở màn hình analytics, dữ liệu lịch sử được truy xuất theo cửa sổ thời gian để phục vụ quan sát xu hướng và đối chiếu khi kiểm thử. Riêng màn hình control panel đảm nhiệm vai trò điều khiển và cấu hình, bao gồm bật hoặc tắt auto-control, điều khiển từng thiết bị ở manual mode và đồng bộ ngưỡng giữa dashboard với firmware.

7.1.4 Luồng điều khiển và truy vết

Khi người dùng gửi lệnh từ giao diện web, frontend gọi REST API tương ứng trên backend. Backend kiểm tra request, chuyển nó thành payload điều khiển theo contract MQTT, publish xuống topic `iot/device/control` và ghi lại lệnh trong `control_log`. Ở chiều ngược lại, ESP32 có thể phản hồi bằng ACK hoặc status qua `iot/device/status`; backend đọc lại phản hồi này để cập nhật trạng thái realtime gần nhất và hỗ trợ hiển thị cho giao diện điều khiển.

Bảng 7.5: Nhóm lệnh điều khiển chính của hệ thống

Lệnh	Vai trò
GET_STATUS, GET_AUTO	Yêu cầu thiết bị trả về trạng thái hiện tại
SET_AUTO	Bật hoặc tắt chế độ điều khiển tự động
SET_DEVICE	Điều khiển từng thiết bị khi hệ thống ở manual mode
GET_THRESHOLDS	Yêu cầu thiết bị publish ngưỡng hiện tại
SET_THRESHOLDS	Cập nhật ngưỡng điều khiển và lưu cấu hình
RESET_THRESHOLDS	Khôi phục cấu hình ngưỡng mặc định
REBOOT	Khởi động lại thiết bị từ xa

Về phía backend, một điều kiện an toàn được áp dụng trước khi gửi lệnh là kiểm tra xem thiết bị có còn online theo telemetry gần nhất hay không. Nếu thiết bị đã offline, backend từ chối publish lệnh mới và ghi log trạng thái thất bại. Cách xử lý này giúp tránh việc giao diện tạo cảm giác điều khiển thành công trong khi ESP32 thực tế không còn kết nối.

7.1.5 Contract dữ liệu giữa các tầng

Toàn bộ luồng tích hợp giữa firmware, backend, cơ sở dữ liệu và dashboard dựa trên contract chung tại `common/data_format.json`. Contract này chuẩn hóa ba nhóm nội dung: telemetry từ ESP32, command từ backend xuống thiết bị và ACK/status theo chiều ngược lại. Nhờ có contract chung, các tầng sử dụng cùng tên topic MQTT, cùng quy ước dữ liệu thiếu và cùng cấu trúc các trường cảnh báo, trạng thái cảm biến và trạng thái thiết bị.

Bảng 7.6: Các nhóm thông tin chính trong contract MQTT/JSON

Nhóm dữ liệu	Nội dung chính
Telemetry	temperature, humidity, pm25, co2, voc, alert_level, sensor_health, alerts, devices
Trạng thái cấu hình	auto_control_enabled, config_version, thresholds
Điều khiển và ACK	command, command_id, status, message

Các giá trị thiếu cũng được giữ thống nhất giữa các tầng: PM2.5 dùng -1, CO₂ dùng 65535, còn nhiệt độ và độ ẩm có thể được biểu diễn bằng null. Backend là nơi tiếp nhận các quy ước này, ánh xạ sang mô hình dữ liệu nội bộ và từ đó cung cấp một API ổn định cho dashboard. Đây là điểm then chốt giúp hệ thống giảm sai lệch giữa firmware, server và giao diện trong quá trình tích hợp.

8. Kết luận và hướng phát triển

8.1 Kết luận

Đề tài đã xây dựng được một nguyên mẫu hệ thống IoT giám sát chất lượng không khí trong phòng học, bao gồm các thành phần chính: nút ESP32 thu thập dữ liệu cảm biến, firmware RTOS xử lý cục bộ, kênh truyền thông MQTT, backend Node.js/Express, cơ sở dữ liệu MySQL và dashboard web phục vụ giám sát, phân tích và điều khiển. Kết quả này cho thấy kiến trúc tổng thể của hệ thống đã được hình thành đầy đủ ở mức nguyên mẫu và các tầng có thể phối hợp với nhau theo một contract dữ liệu thống nhất.

Ở tầng thiết bị, hệ thống đã thu thập được các nhóm đại lượng môi trường quan trọng gồm CO₂, bụi mịn, nhiệt độ, độ ẩm và VOC. Dữ liệu được chuẩn hóa về cùng mô hình, đánh giá bằng cơ chế ngưỡng có hysteresis và ánh xạ sang trạng thái cảnh báo cũng như trạng thái thiết bị xử lý cục bộ. Nhờ đó, thiết bị không chỉ thực hiện chức năng đo lường mà còn có khả năng phản ứng tại chỗ khi điều kiện môi trường vượt khỏi vùng vận hành mong muốn.

Ở tầng hệ thống, firmware RTOS đã được tổ chức thành các tác vụ tương đối độc lập cho đọc cảm biến, xử lý IAQ, điều khiển, cảnh báo và truyền thông MQTT. Tầng backend tiếp nhận telemetry, lưu lịch sử, cung cấp API và hỗ trợ luồng điều khiển ngược từ dashboard đến ESP32. Trên cơ sở đó, dashboard web đã cung cấp được ba nhóm chức năng cốt lõi: quan sát realtime, xem lại dữ liệu lịch sử và thực hiện điều khiển hoặc cấu hình từ xa.

Nhìn chung, đề tài đã chứng minh được tính khả thi của pipeline **Sensor -> ESP32/RTOS -> MQTT -> Backend -> Database -> Dashboard**. Đây là kết quả quan trọng đối với phạm vi đồ án, vì hệ thống không dừng ở mức đọc cảm biến đơn lẻ mà đã đạt tới một mô hình IoT hoàn chỉnh ở mức nguyên mẫu, có dữ liệu thống nhất, có lưu trữ, có giao diện và có khả năng điều khiển hai chiều.

8.2 Giới hạn hiện tại

Mặc dù đã đạt được mục tiêu ở mức nguyên mẫu, hệ thống hiện tại vẫn còn một số giới hạn. Trước hết, các đầu ra điều khiển mới dừng ở mức mô phỏng bằng LED, chưa

thay thế bằng thiết bị chấp hành công suất thực như quạt thông gió, máy lọc không khí hoặc relay điều khiển tải. Vì vậy, các vấn đề về an toàn điện, cách ly công suất và vận hành thiết bị thực chưa nằm trong phạm vi hoàn thiện của phiên bản hiện tại.

Thứ hai, chỉ số VOC đang được sử dụng dưới dạng raw/index tương đối, chưa được hiệu chuẩn thành đại lượng định lượng có ý nghĩa tham chiếu rộng hơn. Tương tự, các ngưỡng điều khiển hiện nay phù hợp cho mục tiêu kiểm thử và minh họa cơ chế vận hành, nhưng chưa thể xem là bộ ngưỡng cuối cùng cho triển khai thực địa nếu chưa có quá trình đo đạc, đối chiếu và hiệu chuẩn trong môi trường sử dụng thực tế.

Thứ ba, hệ thống mới được kiểm chứng tốt ở mức nguyên mẫu kỹ thuật và các bài kiểm thử thành phần. Dù backend đã có test cho MQTT và API, hệ thống vẫn cần được đánh giá end-to-end trong điều kiện vận hành liên tục với ESP32 thực, broker thực, cơ sở dữ liệu và dashboard hoạt động đồng thời trong thời gian dài. Đây là bước cần thiết để đánh giá độ ổn định, độ trễ, khả năng phục hồi kết nối và tỷ lệ mẫu đo hợp lệ.

Cuối cùng, các yêu cầu về bảo mật và quản trị vận hành hiện vẫn ở mức cơ bản. MQTT và API chủ yếu phục vụ môi trường lab hoặc demo; các cơ chế như xác thực người dùng, phân quyền, mã hóa kết nối và audit log vẫn cần được bổ sung nếu hệ thống được mở rộng sang môi trường triển khai thực tế.

8.3 Hướng phát triển

Hướng phát triển đầu tiên là hoàn thiện tầng phần cứng và thiết bị chấp hành. Các LED mô phỏng có thể được thay bằng relay, MOSFET hoặc các module điều khiển phù hợp với quạt thông gió, thiết bị lọc bụi, thiết bị lọc than, điều hòa hoặc máy tạo ẩm. Khi đó, hệ thống sẽ tiến gần hơn tới một mô hình điều khiển IAQ có khả năng ứng dụng thực tế.

Hướng thứ hai là hiệu chuẩn cảm biến và tinh chỉnh bộ ngưỡng vận hành. Việc đối chiếu dữ liệu với thiết bị tham chiếu, thu mẫu trong nhiều điều kiện lớp học khác nhau và đánh giá lại ngưỡng CO₂, PM2.5, VOC, nhiệt độ và độ ẩm sẽ giúp hệ thống có cơ sở vận hành chặt chẽ hơn. Đối với VOC, đây cũng là bước cần thiết để chuyển từ raw/index sang một chỉ số có ý nghĩa ứng dụng cao hơn.

Hướng thứ ba là tăng độ tin cậy và khả năng vận hành dài hạn của firmware RTOS. Các cải tiến có thể bao gồm watchdog, giám sát tài nguyên của từng task, cơ chế lưu đệm dữ liệu khi mất kết nối MQTT và đồng bộ bù khi mạng phục hồi. Những bổ sung này sẽ giúp hệ thống ổn định hơn khi chuyển từ nguyên mẫu sang vận hành liên tục.

Hướng thứ tư là mở rộng backend và dashboard theo nhu cầu sử dụng thực tế. Các khả năng có thể được bổ sung gồm cập nhật realtime bằng WebSocket, thống kê dài hạn, cảnh báo qua email hoặc ứng dụng nhắn tin, và quản lý nhiều thiết bị theo `device_id`. Khi số lượng thiết bị tăng lên, đây sẽ là điều kiện cần để hệ thống chuyển từ mô hình demo đơn thiết bị sang mô hình giám sát phân tán.

Cuối cùng, một hướng phát triển quan trọng là hoàn thiện lớp bảo mật và vận hành. Việc bổ sung xác thực cho API, bảo vệ broker MQTT, quản lý cấu hình thiết bị và hỗ trợ cập nhật firmware sẽ giúp hệ thống phù hợp hơn với yêu cầu triển khai ngoài môi trường phòng thí nghiệm.

Tài liệu tham khảo

- [1] World Health Organization, *WHO guidelines for indoor air quality: selected pollutants*. WHO Regional Office for Europe, 2010.
- [2] United States Environmental Protection Agency, *Indoor Air Quality and Student Performance*. EPA, 2003.
- [3] M. Pulimeno, P. Piscitelli, S. Colazzo, A. Colao, and A. Miani, “Indoor air quality at school and students’ performance,” *Health Promotion Perspectives*, 2020.
- [4] P. Wargocki, J. A. Porras-Salazar, S. Contreras-Espinoza, and W. Bahnfleth, “The relationships between classroom air quality and children’s performance in school,” *Building and Environment*, 2020.
- [5] N. Canha, C. Correia, S. Mendez, C. A. Gamelas, and M. Felizardo, “Monitoring Indoor Air Quality in Classrooms Using Low-Cost Sensors: Does the Perception of Teachers Match Reality?” *Atmosphere*, vol. 15, no. 12, 1450, 2024.
- [6] A. P. Renold, A. A. Abins, and J. Katiravan, “IoT-Driven classroom air quality management with deep hierarchical cluster analysis,” *Scientific Reports*, 2025.
- [7] B. Merema, M. Delwati, M. Sourbron, and H. Breesch, “Demand controlled ventilation in school and office buildings: Lessons learnt from case studies,” *Energy and Buildings*, 2018.
- [8] United States Environmental Protection Agency, *Reference Guide for Indoor Air Quality in Schools*. Truy cập ngày 2026-05-16 tại <https://www.epa.gov/iaq-schools/reference-guide-indoor-air-quality-schools>.
- [9] American Lung Association, *Indoor Air Quality in Schools Guide*. Truy cập ngày 2026-05-16 tại <https://www.lung.org/getmedia/14c45699-8055-4cdc-8695-7a57ae36058a/ALA-IAQ-School1-V2.pdf>.
- [10] World Health Organization, *WHO Global Air Quality Guidelines*. Truy cập ngày 2026-05-16 tại <https://www.who.int/publications/i/item/9789240034228>.

- [11] Espressif Systems, *ESP32 Wi-Fi & Bluetooth SoC*. Truy cập ngày 2026-05-16 tại <https://www.espressif.com/en/products/socs/esp32>.
- [12] OASIS, *MQTT Version 5.0 OASIS Standard*. Truy cập ngày 2026-05-16 tại <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>.
- [13] MQTT.org, *MQTT: The Standard for IoT Messaging*. Truy cập ngày 2026-05-16 tại <https://mqtt.org/>.
- [14] Espressif Systems, *Arduino core for the ESP32*, tài liệu lập trình ESP32 với Arduino framework.
- [15] Modbus Organization, *MODBUS Application Protocol Specification*, tài liệu giao thức Modbus.
- [16] Plantower, *PMS7003 Digital Universal Particle Concentration Sensor Datasheet*.